UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

Concurrencia - Serializabilidad

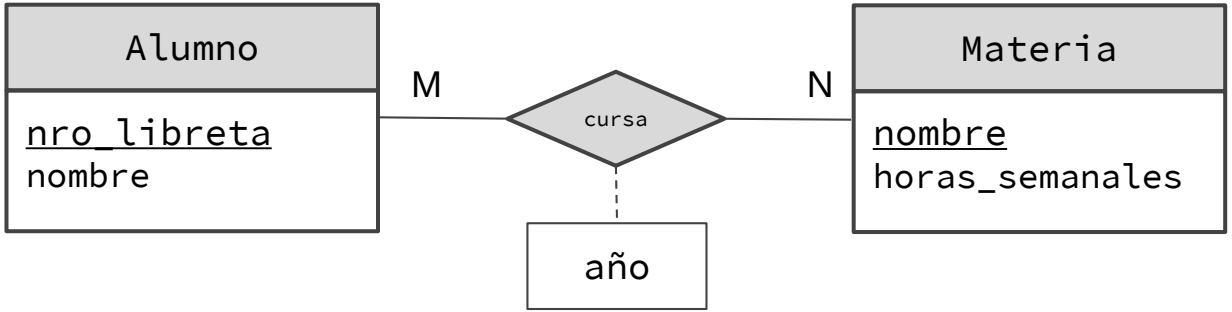
Operaciones SQL y concurrencia

Concurrencia

- Surge la posibilidad de ejecutar múltiples operaciones simultáneas en una base de datos. Poco probable que dos personas quieran acceder a la misma celda, pero pasa
- Esto no debería ser problema con operaciones atómicas, pero cuando las transacciones a realizar consisten de diversas operaciones ordenadas, aparece la necesidad de manejar la **concurrencia**.

Concurrencia

entidades



relaciones

```
Alumnos(nro_libreta, nombre)
Materias(nombre, horas_semanales)
Cursa(libreta_alumno, nombre_materia, año)
```

Concurrencia

- Imaginemos un sistema de inscripción online
- Asumamos que tenemos un cupo en cada materia por el cual no podemos anotar más de 20 alumnos por cursada
- ¿Qué pasa si *simultáneamente* dos alumnos tratan de inscribirse a la misma materia para la cual ya hay 19 inscriptos?

Concurrencia

- Podemos pensar la operación como:

```
if (cant_estudiantes_inscriptos < 20):
    agregar_nuevo_estudiante();
```

- Y esto se traduce en dos queries de SQL:

```
SELECT COUNT(*) FROM cursa WHERE nombre_materia = 'TD7' and año = 2025;
INSERT INTO cursa VALUES (libreta, 'TD7', 2025);
```

- ¿Qué pasa si ambos procesos resuelven el SELECT antes de que ninguno de los dos resuelva el INSERT?

  Van a ver que cant_estudiantes_inscriptos es menor a 20.

Transacciones

Transacciones

- Una **transacción** es una secuencia de operaciones de SQL que se ejecuta como una unidad lógica de trabajo y se comporta de forma indivisible. En las RDBMS deben garantizar las cuatro propiedades *ACID*:
  - **Atomicity**: o todas o ninguna de las operaciones de la transacción se verán reflejadas en la base de datos.
  - **Consistency**: una transacción lleva la base de un estado válido según sus constraints e invariantes a otro igualmente válido.
  - **Isolation**: transacciones ejecutadas concurrentemente producen un resultado equivalente a que se hayan realizado en serie, respetando el orden de inicio.
  - **Durability**: los efectos de una transacción finalizada perduran más allá de ninguna falla de sistema.

Transacciones

- Una transacción se declara de forma explícita a través de comandos SQL:
  - ***begin***: para señalar que los siguientes comandos van a pertenecer a una transacción.
  - ***commit***: indica que la transacción terminó y sus resultados deben hacerse efectivos.
  - ***abort/rollback***: descarta la totalidad de los comandos realizados en la transacción y por lo tanto anula sus efectos.

Serializabilidad

Serializabilidad

- Los motores modernos permiten la ejecución concurrente de múltiples transacciones.
- Para mantener la propiedad de **isolation**, los motores implementan distintos sistemas de control de concurrencia.
- Estos sistemas están basados en el concepto de **serializabilidad:**
  - Si reducimos las posibles operaciones a sólo dos instrucciones atómicas: **read(X)** y **write(X)**
  - Y entendemos X como ítems de datos atómicos (**un** atributo de **una** tupla)
  - **Serializabilidad** es la propiedad de un schedule o solapamiento (secuencia de reads y writes) de transacciones concurrentes de comportarse como si esas transacciones fueran seriales.

Serializabilidad

- Para analizar la serializabilidad usaremos la siguiente notación:
  - **$R_T(X)$**: La transacción **T** lee el ítem **X**.
  - **$W_T(X)$**: La transacción **T** escribe el ítem **X**.
  - **b_T**: Comienza de la transacción **T**.
  - **c_T**: Se hace commit de la transacción **T**.
  - **a_T**: Se aborta la transacción **T**.
- Un **schedule** entre dos transacciones T_1 y T_2 es una lista de instrucciones ordenada donde aparecen todas las instrucciones de T_1 y T_2, respetando los órdenes internos.

Serializabilidad

- Tres nociones de equivalencia:
  - **Resultados:** Dado un estado inicial de la base, ambas historias dejan la base en el mismo estado final.
  - **Conflictos:** Si tienen todas las operaciones en **conflicto** en el mismo orden.
  - **Vistas:** Cuando en cada historia, cada lectura $R_{Ti}(X)$ lee el valor escrito por las mismas transacciones Tj… Tn.

- Nos centramos en la equivalencia de **conflictos** por ser la más presente en las implementaciones.

Serializabilidad

- Definimos conflicto como un par de instrucciones (I_1 e I_2) pertenecientes a transacciones distintas donde:
  - ($R_{Ti}(X)$, $W_{Tj}(X)$): Una transacción escribe un ítem que otra leyó.
  - ($W_{Ti}(X)$, $R_{Tj}(X)$): Una transacción lee un ítem que otra escribió.
  - (WTi(X), WTj(X)): Dos transacciones escriben un mismo ítem.
- Si un par de instrucciones de distintas transacciones I_1 e I_2 no constituyen un conflicto, su orden puede ser invertido obteniendo historias **equivalentes por conflictos**.

Serializabilidad

- Dada una transferencia bancaria de 100 de la cuenta A a la cuenta B:
  - T lee el saldo de la cuenta A: $R_T(A)$
  - T le resta 100 al saldo y lo escribe en la cuenta A: $W_T(A)$
  - T lee el saldo de la cuenta B: $R_T(B)$
  - T le suma 100 al saldo y lo escribe en la cuenta B: $W_T(B)$
- Imaginemos dos transacciones **concurrentes** T_1 y T_2 que ambas realizan una transferencia de A a B.

Serializabilidad

- El schedule serializado sería:

$$R_{T1}(A);\ W_{T1}(A);\ R_{T1}(B);\ W_{T1}(B);\ R_{T2}(A);\ W_{T2}(A);\ R_{T2}(B);\ W_{T2}(B);$$

- Es decir una ejecución que es el resultado de realizar una transacción en su totalidad (T_1) y recién entonces ejecutar la otra (T_2)

Serializabilidad

- Dado un schedule cualquiera, por ejemplo:

$$R_{T1}(A);\ W_{T1}(A);\ R_{T2}(A);\ W_{T2}(A);\ R_{T1}(B);\ W_{T1}(B);\ R_{T2}(B);\ W_{T2}(B);$$

- Debemos determinar si es posible llegar al schedule serializado sólo realizando inversiones que respeten la equivalencia por conflictos. Si esto se cumple, el schedule es **serializable**

Serializabilidad

- Los conflictos serán:
  - $R_{T1}(A)$, $W_{T2}(A)$ T1 lee A que T2 escribió.
  - $W_{T1}(A)$, $R_{T2}(A)$ T1 escribe A que T2 leyó.
  - $W_{T1}(A)$, $W_{T2}(A)$ T1 y T2 escriben en A.
  - $R_{T1}(B)$, $W_{T2}(B)$ T1 lee B que T2 escribió.
  - $W_{T1}(B)$, $R_{T2}(B)$ T1 escribe B que T2 leyó.
  - $W_{T1}(B)$, $W_{T2}(B)$ T1 y T2 escriben en B.

- Es decir que podemos invertir las instrucciones que operan sobre A con las que operan sobre B sin romper la equivalencia

Serializabilidad

- Dado el schedule inicial:

Invertir

$R_{T1}(A);\ W_{T1}(A);\ [R_{T1}(B);\ W_{T1}(B);]\ [R_{T2}(A);\ W_{T2}(A);]\ R_{T2}(B);\ W_{T2}(B);$

- Obtenemos:

$R_{T1}(A);\ W_{T1}(A);\ [R_{T2}(A);\ W_{T2}(A);]\ [R_{T1}(B);\ W_{T1}(B);]\ R_{T2}(B);\ W_{T2}(B);$

- Nuestro schedule es serializable por conflictos

Serializabilidad - Grafo de precedencias

- Una forma más sencilla de determinar la serializabilidad por conflictos es a través de un **grafo de precedencias**.
- Dado un schedule, su grafo tendrá como nodos a todas las transacciones y como ejes (dirigidos) a las relaciones entre T_1 y T_2 siempre que:
  - $R_{T1}(X)$ ocurra antes que $W_{T2}(X)$
  - $W_{T1}(X)$ ocurra antes que $R_{T2}(X)$
  - $W_{T1}(X)$ ocurra antes que $W_{T2}(X)$
- En otras palabras: siempre que haya un **conflicto** entre una instrucción de T_1 y una de T_2 que ocurran en ese orden.

Serializabilidad - Grafo de precedencias

- En un **schedule serializado** entre T_1 y T_2 el único eje posible es $T_1 \rightarrow T_2$ (dado que todas las instrucciones de T_1 ocurren antes que todas las de T_2).
- Esto mismo es cierto para cualquier número de transacciones.
- Un schedule será serializable por conflictos siempre que su grafo de precedencias **no contenga ciclos**.

Serializabilidad - Grafo de precedencias

- ¿Cuál es el grafo de este schedule?

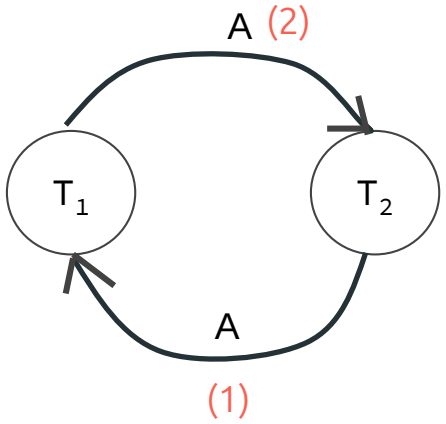
$\underset{(1)}{\underline{R_{T1}(A);}}$ $\underset{(2)}{\underline{R_{T2}(A);}}$ $\underset{(2)}{\underline{W_{T1}(A);}}$ $\underset{(1)}{\underline{W_{T2}(A);}}$ $R_{T1}(B);$ $W_{T1}(B);$ $R_{T2}(B);$ $W_{T2}(B);$

- Nodos: T_1 y T_2
- Ejes:
  - $T_1 \rightarrow T_2$ (por $R_{T1}(A);$ … $W_{T2}(A);$) (2)
  - $T_2 \rightarrow T_1$ (por $R_{T2}(A);$ $W_{T1}(A);$) (1)

Serializabilidad - Grafo de precedencias

- ¿Cuál es el grafo de este schedule?

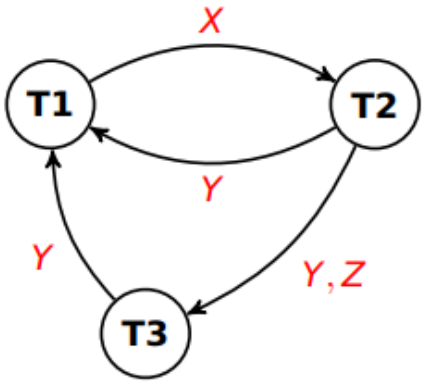
$R_{T1}(A);\ R_{T2}(A);\ W_{T1}(A);\ W_{T2}(A);\ R_{T1}(B);\ W_{T1}(B);\ R_{T2}(B);\ W_{T2}(B);$



Serializabilidad - Grafo de precedencias

- $H = r_2(Z); r_2(Y); w_2(Y); r_3(Y); r_3(Z); r_1(X); w_1(X); w_3(Y); w_3(Z); r_2(X); r_1(Y); w_1(Y); w_2(X)$

T_1	T_2	T_3
	$r_2(Z)$	
	$r_2(Y)$	
	$w_2(Y)$	
		$r_3(Y)$
		$r_3(Z)$
$r_1(X)$		
$w_1(X)$		
		$w_3(Y)$
		$w_3(Z)$
	$r_2(X)$	
$r_1(Y)$		
$w_1(Y)$		
	$w_2(X)$	



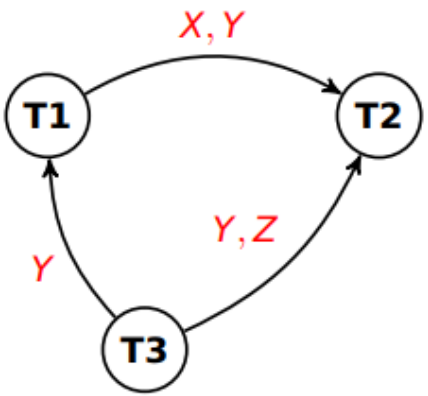
Tiene ciclos, ej:

$T_2 \rightarrow T_3 \rightarrow T_1 \rightarrow T_2$

Serializabilidad - Grafo de precedencias

$H = r_3(Y); r_3(Z); r_1(X); w_1(X); w_3(Y); w_3(Z); r_2(Z); r_1(Y); w_1(Y);$
$r_2(Y); w_2(Y); r_2(X); w_2(X)$

T_1	T_2	T_3
		$r_3(Y)$
		$r_3(Z)$
$r_1(X)$		
$w_1(X)$		
		$w_3(Y)$
		$w_3(Z)$
	$r_2(Z)$	
$r_1(Y)$		
$w_1(Y)$		
	$r_2(Y)$	
	$w_2(Y)$	
	$r_2(X)$	
	$w_2(X)$	



Orden Serial: T3, T1, T2

Vamos sacando cada nodo